

DaSEH R Cheatsheet

Basic R

Major concepts

- **Package:** A bundle of code and/or data that can be loaded for repeated use or sharing, like an expansion pack.
- **Object:** Something that can be worked with in R.
- **Function:** A piece of code that performs a task in R. Functions can come from base R, additional packages, or your own code.
- **Argument:** An option specified in a function.
- **tidyverse:** A set of packages designed for data science that can make code more intuitive than base R.
- **R console:** A full calculator:
 - +, -, /, and * add, subtract, divide, and multiply.
 - ^ or ** raises to a power.
 - Parentheses, (and), control order of operations.
 - %% finds the remainder.
- **Comments:** # is the comment symbol; nothing to its right is evaluated.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code><-</code>	<code>x <- 1</code>	Assigns a name, <code>x</code> , to something in the R environment.
Base R	<code>c()</code>	<code>x <- c(1, 3)</code>	Combines values into a vector.
Base R	<code>str()</code>	<code>str(x)</code>	Summarizes the structure of object <code>x</code> .
Base R	<code>class()</code>	<code>class(x)</code>	Returns the type of values in object <code>x</code> .
Base R	<code>length()</code>	<code>length(x)</code>	Returns the length of object <code>x</code> .
Base R	<code>seq()</code>	<code>seq(from = 0, to = 100, by = 5)</code>	Generates regular sequences.
Base R	<code>rep()</code>	<code>rep(1, times = 10)</code>	Replicates values. Can use the <code>times</code> or <code>length.out</code> argument.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>sample()</code>	<code>sample(1:12)</code>	Samples from values, with or without replacement. <code>replace = TRUE</code> samples with replacement.
Base R	<code>install.packages()</code>	<code>install.packages("tidyverse")</code>	Installs packages.
Base R	<code>library()</code>	<code>library(tidyverse)</code>	Loads and attaches packages to the R environment. Run each time you start R.

RStudio

Major concepts

- **RStudio:** An Integrated Development Environment (IDE) for R that makes R easier to use.
- **Source/Editor:** Where code is written and saved in an `.R` script.
- **R Console:** Where code is executed. Code entered here is not saved on disk.
- **Workspace/Environment:** Shows objects currently loaded in R.
- **R Markdown:** `.Rmd` files that generate reports with code and output.
- **R Project:** Helps organize work, manage working directories, and track the active project.
- **Getting help:** For any function, use `?FUNCTION_NAME` or `help("FUNCTION_NAME")` to view its help file.
- **RStudio keyboard shortcuts:** [RStudio keyboard shortcuts](#)

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>View()</code>	<code>View(mtcars)</code>	Opens data in a spreadsheet-like viewer.
Base R	<code>head()</code>	<code>head(mtcars)</code>	Returns the first 6 rows by default. Use the <code>n</code> argument to specify the number of rows.
Base R	<code>tail()</code>	<code>tail(mtcars)</code>	Returns the last 6 rows by default. Use the <code>n</code> argument to specify the number of rows.

Reproducibility

Major concepts

- **Reproducibility/Reproducible:** A different analyst re-runs the analysis with the same code and data and gets the same result.
- **Repeatable:** The same analysis is repeated under the same conditions and gives the same results.
- **Replicable:** New data are used to test whether the same inferences hold.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>sessionInfo()</code>	<code>sessionInfo()</code>	Returns the R version, OS, and attached packages in the current R session.
Base R	<code>sample()</code>	<code>sample(x, size = 10, replace = FALSE)</code>	Returns a random vector of a specified size, with or without replacement.
Base R	<code>set.seed()</code>	<code>set.seed(1234)</code>	Makes random-number output reproducible for subsequent code.

More resources

- [The R Markdown book](#)
- [Jenny Bryan's organizational strategies](#)
- [Write efficient R code for science](#)
- [Reproducibility in Cancer Informatics course](#)

Data Input

Major concepts

- **Delimited file:** A file in which columns are separated by punctuation, such as commas in CSV files or tabs in TSV/TXT files.
- **Tibble:** A rectangular data frame with data organized into rows and columns.
- **File path:** A file's location on your computer or the internet. [File paths can be relative or absolute.](#)
- **R Projects:** Set the working directory to the folder containing the `.Rproj` file.
- **Reading in data manually:** Download the data and put it in the project folder. In RStudio, go to File > Import Dataset > From Text (`readr`), browse for the file, then click “Update” and “Import.”

Functions

Library/Package	Piece of code	Example of usage	What it does
tidyverse (<code>readr</code>)	<code>read_csv()</code>	<code>df <- read_csv("<url>")</code>	Reads in a CSV file from a specified file path.
tidyverse (<code>readr</code>)	<code>read_tsv()</code>	<code>df <- read_tsv("<url>")</code>	Reads in a TSV file from a specified file path.
tidyverse (<code>readr</code>)	<code>read_delim()</code>	<code>df <- read_delim("<url>", delim = ":")</code>	Reads in a delimited file from a specified file path.
Base R	<code>head()</code> / <code>tail()</code>	<code>head(df)</code> / <code>tail(df)</code>	Returns the first or last part of a dataset or other object.
Base R	<code>getwd()</code>	<code>getwd()</code>	Returns the current working directory.
Base R	<code>setwd()</code>	<code>setwd("Desktop")</code>	Changes the current working directory.
Base R	<code>View()</code>	<code>View(df)</code>	Opens a spreadsheet-style data viewer for a matrix-like R object.
Base R	<code>str()</code>	<code>str(df)</code>	Displays the internal structure of an R object, including dimensions and class.

Subsetting Data in R

Functions

Library/Package	Code	Example	What it does
Base R	<code>nrow(); ncol()</code>	<code>nrow(x); ncol(x)</code>	Get the number of rows and columns in object <code>x</code> , respectively.
Base R	<code>dim()</code>	<code>dim(x)</code>	Get the number of rows and columns in object <code>x</code> .
tidyverse (dplyr)	<code>glimpse()</code>	<code>glimpse(mtcars)</code>	Get an overview of data frame <code>x</code> .
tidyverse (dplyr)	<code>slice_sample()</code>	<code>slice_sample(mtcars)</code>	View a random subset of rows from <code>x</code> .
Base R	<code>data.frame()</code>	<code>df <- data.frame(1:3)</code>	Create a data frame. Named arguments must have the same length.
tidyverse (tibble)	<code>tibble()</code>	<code>tibble(mtcars)</code>	Create a tibble from a data frame or matrix.
tidyverse (tibble)	<code>rownames_to_column()</code>	<code>df <- rownames_to_column(df, "new_col")</code>	Convert row names to a new first column. The argument names the new column.
Base R	<code>colnames()</code>	<code>colnames(df)</code>	Get or set column names for a matrix or data frame.
tidyverse (dplyr)	<code>rename()</code>	<code>df <- rename(df, MPG = mpg)</code>	Rename selected columns while keeping all variables.
tidyverse (dplyr)	<code>rename_with()</code>	<code>df <- rename_with(df, toupper)</code>	Rename selected columns with a function.
Base R	<code>toupper()</code>	<code>toupper("text")</code>	Convert lowercase characters to uppercase.
Base R	<code>tolower()</code>	<code>tolower("TEXT")</code>	Convert uppercase characters to lowercase.
janitor	<code>clean_names()</code>	<code>clean_names(df)</code>	Return a data frame with clean names.
tidyverse (dplyr)	<code>pull()</code>	<code>pull(df, "existing_variable_name")</code>	Extract a column as a vector. Base R notation: <code>df\$column</code> .
tidyverse (dplyr)	<code>select()</code>	<code>select(df, "existing_variable_name")</code>	Select columns that match the specified argument. (<i>See note</i>)
tidyverse (dplyr)	<code>filter()</code>	<code>filter(df, mpg > 20)</code>	Return rows that match the specified logical condition.

Library/Package	Code	Example	What it does
Base R	<code>==, <=, >=, !=</code>	<code>filter(df, mpg > 20)</code>	Compare values in an object. Useful with <code>filter()</code> .
Base R	<code>%in%</code>	<code>filter(df, mpg %in% c(20, 21, 22))</code>	Check whether values on the left are in the object on the right. Returns TRUE or FALSE.
Base R	<code> ></code>	<code>df <- df > select("variable_name")</code>	Pipe a data frame through additional operations.
tidyverse (magrittr)	<code>%>%</code>	<code>df <- df %>% select("variable_name")</code>	Pipe a data frame through tidyverse operations.
tidyverse (dplyr)	<code>mutate()</code>	<code>df <- mutate(df, newcol = wt / 2.2)</code>	Add a new column based on existing columns.
tidyverse (dplyr)	<code>relocate()</code>	<code>df_carb <- relocate(.data = df, wt, .before = mpg)</code>	Reorder columns in a data frame or tibble.
tidyverse (dplyr)	<code>arrange()</code>	<code>df <- arrange(df, mpg)</code>	Reorder rows in ascending order. Use <code>arrange(desc())</code> for descending order.

NOTE: See [tidyselect helpers](#) (`?dplyr_tidy_select`) for handy logical functions to use with `select()`.

Data Summarization

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>min()</code>	<code>min(x)</code>	Returns the minimum value in object <code>x</code> .
Base R	<code>sum()</code>	<code>sum(x)</code>	Returns the sum of values in object <code>x</code> (integer, numeric, or logical).
Base R	<code>mean()</code>	<code>mean(x)</code>	Returns the arithmetic mean of values in object <code>x</code> (integer, numeric, or logical).
Base R	<code>log()</code>	<code>log(x)</code>	Returns the natural logarithm of object <code>x</code> . Use <code>log2(x)</code> for base 2, or specify another base.
Base R	<code>range()</code>	<code>range(x)</code>	Returns the minimum and maximum values in object <code>x</code> .
Base R	<code>sd()</code>	<code>sd(x)</code>	Returns the standard deviation of object <code>x</code> .
Base R	<code>sqrt()</code>	<code>sqrt(x)</code>	Returns the square root of object <code>x</code> .
Base R	<code>quantile()</code>	<code>quantile(x, probs = .5)</code>	Returns sample quantiles for specified probabilities.
Base R	<code>summary()</code>	<code>summary(x)</code>	Returns a summary of values in object <code>x</code> .
tidyverse (dplyr)	<code>pull()</code>	<code>x_vect <- df > pull(x)</code>	Extracts a single column as a vector. Useful before summary functions like <code>mean()</code> and <code>sum()</code> .
tidyverse (dplyr)	<code>summarize()</code>	<code>df <- df > summarize(mean_x = mean(x))</code>	Summarizes values into one or more output values. <code>summarize()</code> and <code>summarise()</code> are synonyms.
tidyverse (dplyr)	<code>distinct()</code>	<code>df > distinct(factor_name)</code>	Displays unique rows from a data frame or tibble.
tidyverse (dplyr)	<code>n_distinct()</code>	<code>x_vect > n_distinct()</code>	Counts unique values or combinations in one or more vectors.
tidyverse (dplyr)	<code>count()</code>	<code>df > count(factor_name)</code>	Counts the number of rows in each group of a factor variable.
tidyverse (dplyr)	<code>group_by()</code>	<code>df > group_by(factor_name)</code>	Groups rows by specified value(s).
Base R	<code>unique()</code>	<code>unique(df)</code>	Returns object <code>x</code> with duplicate elements or rows removed.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>rowSums()</code>	<code>rowSums(df)</code>	Calculates sums for each row.
Base R	<code>colSums()</code>	<code>colSums(df)</code>	Calculates sums for each column.
Base R	<code>rowMeans()</code>	<code>rowMeans(df)</code>	Calculates means for each row.
Base R	<code>colMeans()</code>	<code>colMeans(df)</code>	Calculates means for each column.

NOTE: Many summarizing functions (e.g., `mean()`, `sum()`) have the argument `na.rm = TRUE` to ignore missing data.

Data Classes

Major concepts

- **Character:** Strings or individual characters, quoted.
- **Numeric:** Any real number(s).
- **Double:** A subset of numeric values that can contain decimals.
- **Integer:** Whole number(s).
- **Factor:** Categorical or qualitative variables.
- **Logical:** Values composed of `TRUE` or `FALSE`.
- **Date/POSIXct:** Calendar dates and times.
- **Matrix:** Two-dimensional data where all rows and columns have the same data type.
- **Data frame:** Two-dimensional data where columns can have different data types.
- **List:** Data that can vary in dimensions and contain many data types, including vectors, strings, matrices, models, and other lists.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>class()</code>	<code>class(x)</code>	Returns the class of an object.
Base R	<code>as.numeric()</code>	<code>as.numeric(x)</code>	Coerces object <code>x</code> to numeric class. Similar functions include <code>as.character()</code> , <code>as.data.frame()</code> , <code>as.matrix()</code> , and <code>as.Date()</code> . Use with <code>mutate()</code> .
tidyverse (lubridate)	<code>ymd()</code>	<code>ymd("2024-01-31")</code>	Coerces character object <code>x</code> to date class. Use functions such as <code>mdy()</code> or <code>dmy()</code> for other date formats.

NOTE: `lubridate` is a powerful, widely used R package from `tidyverse` family to work with Date / POSIXct class objects.

Data Cleaning

Major concepts

- **Most important rule of data handling:** Always look at your data.
- **NA:** General missing data.
- **NaN:** “Not a Number”; occurs when you calculate 0/0.
- **Inf and -Inf:** Infinity; occurs when you divide a positive or negative number by 0.
- **naniar:** Useful for exploring missingness in large datasets. See the [naniar package](#).
- **tidyverse (stringr):** Useful for detecting and working with parts of character values, especially with `filter()` and `str_detect()`.

Examples

Example of `case_when()` with `mutate()`:

```
Orange |>
  mutate(old = case_when(
    age > 1000 ~ TRUE,
    .default = FALSE
  ))
```

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>is.na()</code>	<code>is.na(x)</code> <code>any(is.na(x))</code>	Checks if <code>x</code> is NA. With <code>any()</code> , checks if any values are NA.
Base R	<code>is.nan()</code>	<code>is.nan(x)</code>	Checks if <code>x</code> is NaN.
Base R	<code>is.infinite()</code>	<code>is.infinite(x)</code>	Checks if <code>x</code> is Inf or -Inf.
naniar	<code>pct_complete()</code>	<code>pct_complete(x)</code>	Reports the percentage of complete data in <code>x</code> .
naniar	<code>gg_miss_var()</code>	<code>gg_miss_var(x)</code>	Plots the percentage of missing data by variable in <code>x</code> .
naniar	<code>miss_var_summary()</code>	<code>miss_var_summary(x)</code>	Reports the percentage of missing data by variable in <code>x</code> .
naniar	<code>miss_var_which()</code>	<code>miss_var_which(x)</code>	Returns variable names that contain missing values in <code>x</code> .
tidyverse (tidyr)	<code>drop_na(df)</code>	<code>drop_na(df)</code> <code>drop_na(variable)</code>	Drops rows with NA values. When a variable is specified, only drops rows where that variable is NA.

Library/Package	Piece of code	Example of usage	What it does
tidyverse (dplyr)	<code>na_if()</code>	<code>na_if(vector, value)</code>	Converts specified values to NA. Use with <code>mutate()</code> .
tidyverse (tidyr)	<code>replace_na()</code>	<code>replace_na(vector, value)</code>	Replaces NA values with a specified value. Use with <code>mutate()</code> .
tidyverse (dplyr)	<code>case_when()</code>	<code>case_when(test ~ outcome, .default = x)</code>	Vectorizes multiple <code>if_else()</code> statements. If no cases match, NA is returned unless <code>.default</code> is specified. Use with <code>mutate()</code> .
tidyverse (dplyr)	<code>mutate()</code>	<code>df <- mutate(df, newcol = wt / 2.2)</code>	Adds a new column based on existing columns.
tidyverse (tidyr)	<code>separate()</code>	<code>df > separate(x, c("A", "B"))</code>	Separates one character column into multiple columns using a regular expression or numeric locations.
tidyverse (tidyr)	<code>unite()</code>	<code>df > unite("z", x:y, remove = FALSE)</code>	Unites multiple columns into one column.
tidyverse (stringr)	<code>str_detect()</code>	<code>df > filter(str_detect(col_name, "string_pattern"))</code>	Returns a logical vector indicating whether a string pattern was detected.
tidyverse (stringr)	<code>str_replace()</code>	<code>str_replace(string = vector, "replace_me", "with_me")</code>	Replaces the first instance of one specified string with another.
tidyverse (stringr)	<code>str_replace_all()</code>	<code>str_replace_all(string = vector, "replace_me", "with_me")</code>	Replaces all instances of one specified string with another.
tidyverse (stringr)	<code>str_sub()</code>	<code>str_sub(string = vector, start = 1, end = 3)</code>	Subsets a string to the specified character positions.

Data Manipulation

Major concepts

- **Wide data:** Multiple columns per individual, with values spread across columns
- **Long data:** Multiple rows per observation, with values stored in a single column

Functions

Library/Package	Piece of code	Example of usage	What it does
tidyverse (tidyr)	<code>separate()</code>	<code>df > separate(x, c("A", "B"))</code>	Separates a character column into multiple columns using a regular expression or numeric positions
tidyverse (tidyr)	<code>unite()</code>	<code>df > unite("z", x:y, remove = FALSE)</code>	Combines multiple columns into one column
tidyverse (tidyr)	<code>pivot_longer()</code>	<code>df > pivot_longer(!col_to_keep, names_to = "new_col_with_labels", values_to = "new_col_with_values")</code>	Lengthens a data frame by increasing rows and decreasing columns
tidyverse (tidyr)	<code>pivot_wider()</code>	<code>df > pivot_wider(names_from = "col_with_names", values_from = "col_with_values")</code>	Widens a data frame by decreasing rows and increasing columns
tidyverse (dplyr)	<code>?_join()</code>	<code>inner_join(x, y)</code>	Joins data from two data frames. <code>inner_join(x, y)</code> Keeps only rows that match in both x and y. <code>full_join(x, y)</code> Keeps all rows from x and y. <code>left_join(x, y)</code> Keeps all rows from x, even if they do not match y. <code>right_join(x, y)</code> Keeps all rows from y, even if they do not match x. <code>anti_join(x, y)</code> Keeps rows from x that do not match y, retaining only columns from x.

Data Visualization with **esquisse**

Major concepts

- **esquisse**: Package for testing and learning the syntax for making plots

Functions

Library/Package	Piece of code	Example of usage	What it does
esquisse	esquisser()	<code>esquisser(Orange)</code> <code>esquisser(Orange, viewer = "browser")</code>	Starts an interactive esquisse session to create a plot from a <code>data.frame</code> or <code>tibble</code>

Data Visualization

Major concepts

- **tidyverse (ggplot2):** Produces graphics.
- **Tidy data:** Each variable is a column, and each observation is a row.
 - Having data in tidy format and long format makes creating plots easier!
- **Plot layers:** Use + at the end of each line when adding layers to plots.
- **Pipes:** |> and %>% do not add plot layers, but they can pipe data into plots.
 - **Works:** `ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_boxplot()`
 - **Works:** `iris |> ggplot(aes(x = Species, y = Petal.Length)) + geom_boxplot()`
 - **Does not work:** `iris |> ggplot(aes(x = Species, y = Petal.Length)) |> geom_plot()`

Functions

Library/Package	Piece of code	Example of usage	What it does
tidyverse (ggplot2)	<code>ggplot()</code>	<code>ggplot(data = iris)</code>	Begins a plot that is finished by adding layers.
tidyverse (ggplot2)	<code>aes()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length))</code>	Maps variables to visual properties in a <code>ggplot()</code> object.
tidyverse (ggplot2)	<code>geom_boxplot()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_boxplot()</code>	Creates a boxplot when added as a layer to a <code>ggplot()</code> object.
tidyverse (ggplot2)	<code>geom_density()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density()</code>	Creates a smoothed density plot when added as a layer to a <code>ggplot()</code> object.
tidyverse (ggplot2)	<code>geom_point()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_point()</code>	Creates a scatterplot when added as a layer to a <code>ggplot()</code> object.
tidyverse (ggplot2)	<code>geom_line()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_line()</code>	Creates a line plot by connecting points in order of the x-axis variable.
tidyverse (ggplot2)	<code>geom_hline()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_point() + geom_hline(yintercept = 4)</code>	Adds horizontal lines to a plot.

Library/Package	Piece of code	Example of usage	What it does
tidyverse (ggplot2)	<code>theme_classic()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + theme_classic()</code>	Displays a <code>ggplot()</code> object without grid lines. See the ggtheme documentation for additional themes.
tidyverse (ggplot2)	<code>xlab(); ylab(); ggtitle()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + xlab("Petal Length")</code>	Modifies the x-axis label, y-axis label, and plot title, respectively.
tidyverse (ggplot2)	<code>facet_grid()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + facet_grid(~Species)</code>	Creates a grid of plots using specified variables to subset the data.
tidyverse (ggplot2)	<code>facet_wrap()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + facet_wrap(~Species, scales = "free", nrow = 2)</code>	Creates individual plots using specified variables to subset the data, with flexible layouts and axis scales.
tidyverse (ggplot2)	<code>ggsave()</code>	<code>ggsave(filename = "plotname.pdf")</code>	Saves the last plot in the working directory.

NOTE: Click [here](#) for a summary of the `ggplot2` theme system.

Factors

Major concepts

- **Factor:** A special character vector whose elements have predefined groups, or levels. Factors are useful for qualitative or categorical variables, such as grade levels. Specify factors with the `factor` function so R recognizes them correctly.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>factor()</code>	<code>fact_qual <- factor(c("poor", "fine", "good"))</code>	Creates a factor from a vector. Levels are ordered alphanumerically by default.
Base R	<code>levels()</code>	<code>levels(fact_qual)</code>	Shows the levels and order of a factor vector.
tidyverse (forcats)	<code>as_factor()</code>	<code>fact_qual <- as_factor(c("poor", "fine", "good"))</code>	Creates a factor from a vector. Levels are ordered by first appearance in the data by default.
tidyverse (forcats)	<code>fct_reorder()</code>	<code>ggplot(iris, aes(x = fct_reorder(Species, Sepal.Width, mean), y = Sepal.Width)) + geom_boxplot()</code>	Reorders factor levels according to another variable. Here, <code>Species</code> is ordered by the mean of <code>Sepal.Width</code> .

Statistics

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>cor()</code>	<code>cor(x, y)</code>	Calculates correlation between two vectors.
<code>corrplot</code>	<code>corrplot()</code>	<code>corrplot(cor_mat, type = "upper", order = "hclust")</code>	Creates a correlation matrix plot.
Base R	<code>t.test()</code>	<code>t.test(x, y, alternative = "two.sided")</code>	Performs one- and two-sided t-tests.
<code>broom</code>	<code>tidy()</code>	<code>tidy(t_test_result)</code>	Converts statistical objects to tidy data frames.
Base R	<code>wilcox.test()</code>	<code>wilcox.test(x, y)</code>	Performs nonparametric Wilcoxon signed-rank or rank-sum tests.
Base R	<code>shapiro.test()</code>	<code>shapiro.test(x)</code>	Tests normality with the Shapiro-Wilk test.
Base R	<code>ks.test()</code>	<code>ks.test(x)</code>	Tests a distribution with the Kolmogorov-Smirnov test.
Base R	<code>var.test()</code>	<code>var.test(x, y)</code>	Compares two variances with Fisher's F-test.
Base R	<code>chisq.test()</code>	<code>chisq.test(x, y)</code>	Performs chi-squared contingency table and goodness-of-fit tests.
Base R	<code>summary()</code>	<code>summary(linear_model_result)</code>	Summarizes an object, including models and statistical test results.
Base R	<code>glm()</code>	<code>glm(x ~ y, data = df, family = binomial())</code>	Fits generalized linear models; specify distribution and link with <code>family</code> .
<code>epitools</code>	<code>oddsratio()</code>	<code>oddsratio(x, y)</code>	Calculates odds ratios.

Data Output

Functions

Library/Package	Piece of code	Example of usage	What it does
tidyverse (readr)	<code>write_csv()</code>	<code>write_csv(df, "file.csv")</code>	Writes a data frame to a CSV file.
tidyverse (readr)	<code>write_delim()</code>	<code>write_delim(df, "file.txt", delim = ":")</code>	Writes a data frame to a delimited file.
tidyverse (readr)	<code>write_rds()</code>	<code>write_rds(df, "file.rds")</code>	Saves a single R object to an RDS file.
tidyverse (readr)	<code>read_rds()</code>	<code>df <- read_rds("file.rds")</code>	Reads a single R object from an RDS file.
Base R	<code>save()</code>	<code>save(df1, df2, file = "file.RData")</code>	Saves one or more R objects to an RData file.
Base R	<code>load()</code>	<code>load("file.RData")</code>	Loads R objects from an RData file into the environment.

Functions

Major concepts

- **Create functions to:**
 - Do custom actions
 - Reduce repetitive code
 - Organize code into manageable chunks
 - Avoid running code unintentionally
 - Use names that make sense to you
- **Custom function syntax:**

```
function_name <- function(inputs) {  
  # Document inputs here  
  <function body>  
  return(value)  
}
```

Examples

Example of an **anonymous function** used with `across()`:

```
iris |>  
  summarize(across(  
    starts_with("Sepal"),  
    \(x) mean(x, na.rm = TRUE)  
  ))
```

Functions

Library/package	Piece of code	Example of usage	What it does
Base R	<code>function()</code> or <code>\()</code>	<code>div_100 <- function(x) x / 100</code> or <code>div_100 <- \(x) x / 100</code>	Creates a function; here, <code>x</code> is divided by 100.
Base R	<code>lapply()</code>	<code>lapply(some_list, a_function)</code>	Applies a function over a list or vector.
Base R	<code>sapply()</code>	<code>sapply(some_list, a_function)</code>	Applies a function over a list or vector; returns a matrix or vector by default.

Library/package	Piece of code	Example of usage	What it does
tidyverse (dplyr)	<code>across()</code>	<pre>iris > mutate(across(contains("Sepal"), round)) or iris > group_by(Species) > summarise(across(starts_with("Sepal"), \(x) mean(x, na.rm = TRUE)))</pre>	Applies a function across columns in a data frame.
tidyverse (dplyr)	<code>mutate_if()</code>	<pre>iris > mutate_if(is.numeric, round)</pre>	Applies a function to selected columns in a data frame.

* This cheatsheet format was adapted from Alex's Lemonade Stand materials ([source](#)).

* Find more resources at <https://daseh.org>.